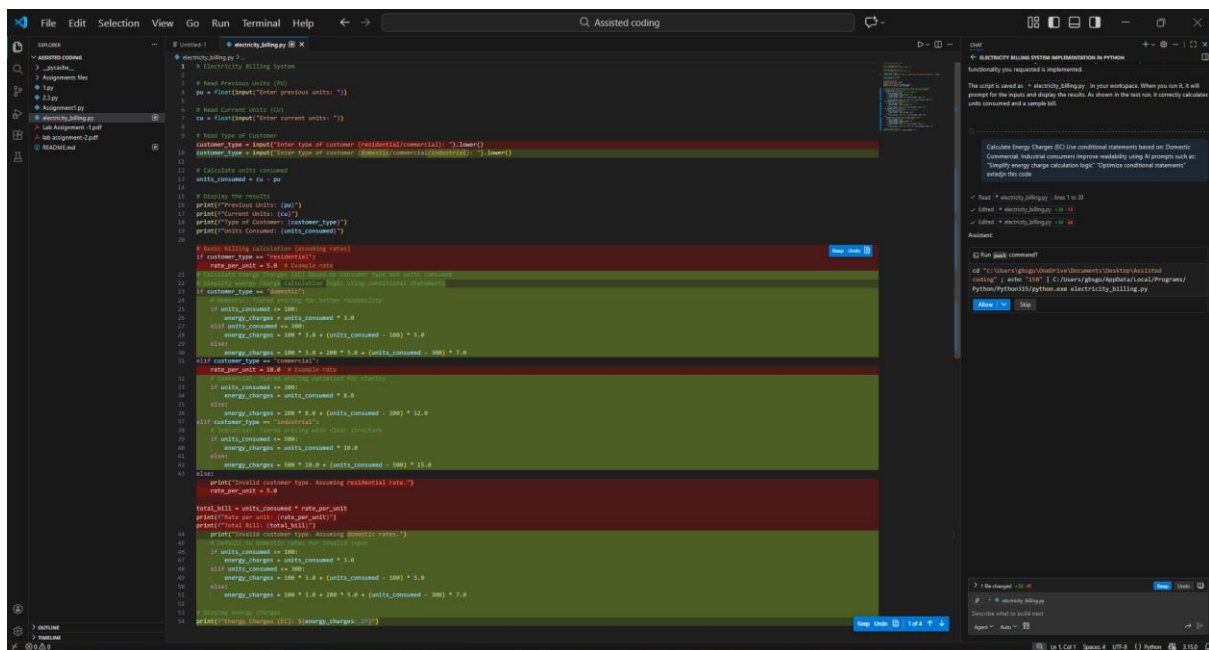




The screenshot shows a VS Code editor with a Python script named `electricity_billing.py`. The script implements a billing system with the following logic:

- Inputs:** `previous_units` (float), `current_units` (float), and `customer_type` (string: 'domestic', 'commercial', 'industrial').
- Units Consumed:** Calculated as `current_units - previous_units`.
- Energy Charges (EC):** Calculated based on customer type and units consumed:
 - Domestic:** Tiered pricing: 5.0 for first 100 units, 5.0 for next 200 units, 7.0 thereafter.
 - Commercial:** Tiered pricing: 8.0 for first 200 units, 12.0 thereafter.
 - Industrial:** Tiered pricing: 10.0 for first 500 units, 15.0 thereafter.
- Fixed Charges (FC):** A constant value of 20.00.
- Output:** A formatted string showing the bill details: `Electricity Bill`, `Previous Units`, `Current Units`, `Customer Type`, `Energy Charges`, `Fixed Charges`, and `Total Bill`.

AI assistance is visible in the right-hand pane, providing comments and suggestions for the code.

Task 3: Modular Design Using AI Assistance (Using Functions)

Scenario

Billing logic must be reusable for multiple consumers.

The screenshot shows a VS Code editor with a Python script named `33.py` that implements a modular electricity billing system using functions. The script is structured as follows:

- Imports:** `def calculate_energy_charges(units_consumed, customer_type):` and `def calculate_fixed_charges(customer_type):`.
- calculate_energy_charges:** A function that calculates energy charges based on customer type and units consumed. It uses tiered pricing:
 - Domestic:** 5.0 for first 100 units, 5.0 for next 200 units, 7.0 thereafter.
 - Commercial:** 8.0 for first 200 units, 12.0 thereafter.
 - Industrial:** 10.0 for first 500 units, 15.0 thereafter.
- calculate_fixed_charges:** A function that returns a fixed charge based on customer type:
 - Domestic:** 20.00
 - Commercial:** 25.00
 - Industrial:** 30.00
- main:** A function that handles user input, calls the calculation functions, and displays the results in a formatted string.

AI assistance is visible in the right-hand pane, providing comments and suggestions for the code.

```
def calculate_energy_charges(units_consumed, customer_type):
    """Calculate energy charges based on units consumed and customer type.
    Returns a tuple of (energy_charges, fixed_charges)."""
    if customer_type == "domestic":
        return units_consumed * 3.0
    elif units_consumed <= 300:
        return 100 * 3.0 + (units_consumed - 100) * 5.0
    else:
        return 100 * 3.0 + 200 * 5.0 + (units_consumed - 300) * 7.0

def calculate_fixed_charges(customer_type):
    """Calculate fixed charges based on customer type.
    Returns a tuple of (energy_charges, fixed_charges)."""
    if customer_type == "domestic":
        return 10.0
    elif customer_type == "commercial":
        return 100.0
    elif customer_type == "industrial":
        return 200.0
    else:
        return 10.0

def main():
    """Main function to handle user input and display billing information.
    Calls the calculation functions and returns the calculated values.
    """
    # Read Previous Units (PU)
    pu = float(input("Enter previous units: "))
    # Read Current Units (CU)
    cu = float(input("Enter current units: "))
    # Read Type of Customer
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

    # Calculate energy charges
    energy_charges = calculate_energy_charges(cu - pu, customer_type)
    # Calculate fixed charges
    fixed_charges = calculate_fixed_charges(customer_type)
    # Calculate total bill
    total_bill = energy_charges + fixed_charges

    # Display the results
    print("\n--- Electricity Bill ---")
    print(f"Previous Units: {pu}")
    print(f"Current Units: {cu}")
    print(f"Units Consumed: {cu - pu}")
    print(f"Customer Type: {customer_type}")
    print(f"Energy Charges: {energy_charges}")
    print(f"Fixed Charges: {fixed_charges}")
    print(f"Total Bill: {total_bill}")

    # Return calculated values as a tuple
    return energy_charges, fixed_charges, total_bill

# Run the main function if this script is executed directly
if __name__ == "__main__":
    main()
```

Terminal Output:

```
PS C:\Users\gmgpu\OneDrive\Documents\Desktop\Assisted coding> .\3.py
--- Electricity Bill ---
Previous Units: 10.0
Current Units: 12.0
Units Consumed: 2.0
Customer Type: Commercial
Energy Charges: $16.00
Fixed Charges: $100.00
Total Bill: $116.00
```

```
def main():
    """Main function to handle user input and display billing information.
    Calls the calculation functions and returns the calculated values.
    """
    # Read Previous Units (PU)
    pu = float(input("Enter previous units: "))
    # Read Current Units (CU)
    cu = float(input("Enter current units: "))
    # Read Type of Customer
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

    # Calculate units consumed
    units_consumed = cu - pu

    # Calculate charges using functions
    energy_charges = calculate_energy_charges(units_consumed, customer_type)
    fixed_charges = calculate_fixed_charges(customer_type)

    # Calculate total bill
    total_bill = energy_charges + fixed_charges

    # Display the results
    print("\n--- Electricity Bill ---")
    print(f"Previous Units: {pu}")
    print(f"Current Units: {cu}")
    print(f"Units Consumed: {units_consumed}")
    print(f"Customer Type: {customer_type}")
    print(f"Energy Charges: {energy_charges}")
    print(f"Fixed Charges: {fixed_charges}")
    print(f"Total Bill: {total_bill}")

    # Return calculated values as a tuple
    return energy_charges, fixed_charges, total_bill

# Run the main function if this script is executed directly
if __name__ == "__main__":
    main()
```

Terminal Output:

```
PS C:\Users\gmgpu\OneDrive\Documents\Desktop\Assisted coding> .\3.py
--- Electricity Bill ---
Previous Units: 10.0
Current Units: 12.0
Units Consumed: 2.0
Customer Type: Commercial
Energy Charges: $16.00
Fixed Charges: $100.00
Total Bill: $116.00
```

Task 4: Calculation of Additional Charges

Scenario

Electricity bills include multiple additional charges.

```
File Edit Selection View Go Run Terminal Help
electric_billings.py
# Electric Billings System - Enhanced Calculations
# Read previous units (pu)
pu = float(input("Enter previous units: "))

# Read current units (cu)
cu = float(input("Enter current units: "))

# Read type of customer
customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

# Calculate units consumed
units_consumed = cu - pu

# Display the results
print("\nPrevious units: {pu:7.2f}")
print("Current units: {cu:7.2f}")
print("Type of customer: {customer_type}")
print("Units Consumed: {units_consumed:7.2f}")

# Calculate Energy Charges (EC) based on customer type and units consumed
# Simplify energy charge calculation logic using conditional statements
if customer_type == "domestic":
    # Domestic: Linear pricing for better readability
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    elif units_consumed > 100:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0
    else:
        energy_charges = 100 * 3.0 + 200 * 5.0 * (units_consumed - 100) * 7.0
elif customer_type == "commercial":
    # Commercial: Linear pricing maintained for clarity
    if units_consumed <= 100:
        energy_charges = units_consumed * 5.0
    else:
        energy_charges = 100 * 5.0 + (units_consumed - 100) * 12.0
elif customer_type == "industrial":
    # Industrial: Linear pricing with clear structure
    if units_consumed <= 100:
        energy_charges = units_consumed * 10.0
    else:
        energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
else:
    print("Invalid customer type. Assuming domestic rates.")
    # Default to domestic rates for invalid input
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    elif units_consumed > 100:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0

# Calculate Fixed Charges (FC) based on customer type
if customer_type == "domestic":
    fixed_charges = 50.0 # Monthly service fee for domestic customers
elif customer_type == "commercial":
    fixed_charges = 100.0 # Monthly service fee for commercial customers
elif customer_type == "industrial":
    fixed_charges = 200.0 # Monthly service fee for industrial customers
else:
    fixed_charges = 50.0 # Default to domestic fixed charges

# Calculate Electricity Duty (ED) as percentage of energy charges (EC)
# Electricity duty rate varies by customer type for improved billing accuracy
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial
else:
    electricity_duty_rate = 0.05 # Default 5%

electricity_duty = energy_charges * electricity_duty_rate

# Calculate Customer Charges (CC) - Total charges including all components
customer_charges = energy_charges + fixed_charges + electricity_duty

# Display the results
print("\n--- Electricity Bill Details ---")
print("Energy Charges (EC): $30.00")
print("Fixed Charges (FC): $50.00")
print("Electricity Duty (ED): $1.50 (5%)")
print("Customer Charges (CC): $81.50
```

```
File Edit Selection View Go Run Terminal Help
electric_billings.py
# Electric Billings System - Enhanced Calculations
# Read previous units (pu)
pu = float(input("Enter previous units: "))

# Read current units (cu)
cu = float(input("Enter current units: "))

# Read type of customer
customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower()

# Calculate units consumed
units_consumed = cu - pu

# Display the results
print("\nPrevious units: {pu:7.2f}")
print("Current units: {cu:7.2f}")
print("Type of customer: {customer_type}")
print("Units Consumed: {units_consumed:7.2f}")

# Calculate Energy Charges (EC) based on customer type and units consumed
# Simplify energy charge calculation logic using conditional statements
if customer_type == "domestic":
    # Domestic: Linear pricing for better readability
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    elif units_consumed > 100:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0
    else:
        energy_charges = 100 * 3.0 + 200 * 5.0 * (units_consumed - 100) * 7.0
elif customer_type == "commercial":
    # Commercial: Linear pricing maintained for clarity
    if units_consumed <= 100:
        energy_charges = units_consumed * 5.0
    else:
        energy_charges = 100 * 5.0 + (units_consumed - 100) * 12.0
elif customer_type == "industrial":
    # Industrial: Linear pricing with clear structure
    if units_consumed <= 100:
        energy_charges = units_consumed * 10.0
    else:
        energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
else:
    print("Invalid customer type. Assuming domestic rates.")
    # Default to domestic rates for invalid input
    if units_consumed <= 100:
        energy_charges = units_consumed * 3.0
    elif units_consumed > 100:
        energy_charges = 100 * 3.0 + 3.0 * (units_consumed - 100) * 5.0

# Calculate Fixed Charges (FC) based on customer type
if customer_type == "domestic":
    fixed_charges = 50.0 # Monthly service fee for domestic customers
elif customer_type == "commercial":
    fixed_charges = 100.0 # Monthly service fee for commercial customers
elif customer_type == "industrial":
    fixed_charges = 200.0 # Monthly service fee for industrial customers
else:
    fixed_charges = 50.0 # Default to domestic fixed charges

# Calculate Electricity Duty (ED) as percentage of energy charges (EC)
# Electricity duty rate varies by customer type for improved billing accuracy
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial
else:
    electricity_duty_rate = 0.05 # Default 5%

electricity_duty = energy_charges * electricity_duty_rate

# Calculate Customer Charges (CC) - Total charges including all components
customer_charges = energy_charges + fixed_charges + electricity_duty

# Display the results
print("\n--- Electricity Bill Details ---")
print("Energy Charges (EC): $30.00")
print("Fixed Charges (FC): $50.00")
print("Electricity Duty (ED): $1.50 (5%)")
print("Customer Charges (CC): $81.50
```

Task 5: Final Bill Generation and Output Analysis

Scenario

The final electricity bill must present all values clearly.

```
File Edit Selection View Go Run Terminal Help
electricity_billings.py
# Electricity Billing System with Enhanced Accuracy and Real-world Features
# This program calculates comprehensive electricity bills with multiple charge components.
# It includes validation and error handling for real-world applicability.

try:
    # Input validation and error handling for real-world applicability
    # Read previous units (0 for no validation)
    prev = float(input("Enter previous units: "))
    if prev < 0:
        raise ValueError("Previous units cannot be negative")

    # Read current units (00 for no validation)
    curr = float(input("Enter current units: "))
    if curr < 0:
        raise ValueError("Current units cannot be negative")

    # Read type of customer with validation
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower().strip()
    valid_types = ["domestic", "commercial", "industrial"]
    if customer_type not in valid_types:
        print(f"Invalid customer type '{customer_type}'. Valid options: {', '.join(valid_types)}")
        customer_type = "domestic" # Default fallback
        print("Using default customer type: {customer_type}")

except ValueError as e:
    print(f"❌ Input Error: {e}")
    print("Please enter valid numeric values for units.")
    exit()

# Calculate units consumed with validation
units_consumed = curr - prev

# Handle negative consumption (possible meter reset or error)
if units_consumed < 0:
    print(f"⚠️ Warning: Current units are less than previous units.")
    print("This may indicate a meter reading error or meter reset.")
    print("Setting energy charges to $0.00 for this billing cycle.")
    energy_charges = 0.0
    electricity_duty = 0.0
    units_consumed_display = f"({units_consumed:1f}) (adjusted to 0 for billing)"
else:
    units_consumed_display = f"{units_consumed:1f}"
    # Calculate Energy Charges (EC) based on customer type and units consumed
    # Standard pricing structure for accurate billing
    if customer_type == "domestic":
        # Domestic tiered pricing: 0.0 for first 100, 0.0 for next 200, 7.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0
    elif customer_type == "commercial":
        # Commercial tiered pricing: 0.0 for first 100, 11.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 11.0
    elif customer_type == "industrial":
        # Industrial tiered pricing: 10.0 for first 100, 15.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
    else:
        # Default to domestic rates for invalid input
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
electricity_duty = electricity_duty_rate * energy_charges

# Bill Summary for Industrial Customer
prev_units = 10
curr_units = 30
customer_type = "Industrial"
units_consumed = 2.0

# Meter Reading Summary
print("--- Meter Reading Summary ---")
print(f"Previous units: {prev_units}")
print(f"Current units: {curr_units}")
print(f"Customer Type: {customer_type}")
print(f"Units Consumed: {units_consumed}")

# Electricity Bill Details
print("--- Electricity Bill Details ---")
print(f"Energy Charges (EC): $30.00")
print(f> Fixed Charges (FC): $10.00")
print(f"Customer Charges (CC): $0.00")
print(f"Electricity Duty (ED): $1.60 (5%)")
print(f>Total Bill Amount: $21.60")

# Bill Summary for Industrial Customer
print("--- Bill Summary for Industrial Customer ---")
print("New structure: Tiered pricing applied")
```

```
File Edit Selection View Go Run Terminal Help
electricity_billings.py
# Electricity Billing System with Enhanced Accuracy and Real-world Features
# This program calculates comprehensive electricity bills with multiple charge components.
# It includes validation and error handling for real-world applicability.

try:
    # Input validation and error handling for real-world applicability
    # Read previous units (0 for no validation)
    prev = float(input("Enter previous units: "))
    if prev < 0:
        raise ValueError("Previous units cannot be negative")

    # Read current units (00 for no validation)
    curr = float(input("Enter current units: "))
    if curr < 0:
        raise ValueError("Current units cannot be negative")

    # Read type of customer with validation
    customer_type = input("Enter type of customer (domestic/commercial/industrial): ").lower().strip()
    valid_types = ["domestic", "commercial", "industrial"]
    if customer_type not in valid_types:
        print(f"Invalid customer type '{customer_type}'. Valid options: {', '.join(valid_types)}")
        customer_type = "domestic" # Default fallback
        print("Using default customer type: {customer_type}")

except ValueError as e:
    print(f"❌ Input Error: {e}")
    print("Please enter valid numeric values for units.")
    exit()

# Calculate units consumed with validation
units_consumed = curr - prev

# Handle negative consumption (possible meter reset or error)
if units_consumed < 0:
    print(f"⚠️ Warning: Current units are less than previous units.")
    print("This may indicate a meter reading error or meter reset.")
    print("Setting energy charges to $0.00 for this billing cycle.")
    energy_charges = 0.0
    electricity_duty = 0.0
    units_consumed_display = f"({units_consumed:1f}) (adjusted to 0 for billing)"
else:
    units_consumed_display = f"{units_consumed:1f}"
    # Calculate Energy Charges (EC) based on customer type and units consumed
    # Standard pricing structure for accurate billing
    if customer_type == "domestic":
        # Domestic tiered pricing: 0.0 for first 100, 0.0 for next 200, 7.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0
    elif customer_type == "commercial":
        # Commercial tiered pricing: 0.0 for first 100, 11.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 11.0
    elif customer_type == "industrial":
        # Industrial tiered pricing: 10.0 for first 100, 15.0 thereafter
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        else:
            energy_charges = 100 * 10.0 + (units_consumed - 100) * 15.0
    else:
        # Default to domestic rates for invalid input
        if units_consumed <= 100:
            energy_charges = units_consumed * 0.0
        elif units_consumed <= 300:
            energy_charges = 100 * 0.0 + (units_consumed - 100) * 0.0
        else:
            energy_charges = 100 * 0.0 + 200 * 0.0 + (units_consumed - 300) * 7.0

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
if customer_type == "domestic":
    electricity_duty_rate = 0.05 # 5% for domestic
elif customer_type == "commercial":
    electricity_duty_rate = 0.08 # 8% for commercial
elif customer_type == "industrial":
    electricity_duty_rate = 0.10 # 10% for industrial

# Calculate Electricity Duty (ED) as percentage of Energy Charges (EC)
electricity_duty = electricity_duty_rate * energy_charges

# Bill Summary for Industrial Customer
prev_units = 10
curr_units = 30
customer_type = "Industrial"
units_consumed = 2.0

# Meter Reading Summary
print("--- Meter Reading Summary ---")
print(f"Previous units: {prev_units}")
print(f"Current units: {curr_units}")
print(f"Customer Type: {customer_type}")
print(f"Units Consumed: {units_consumed}")

# Electricity Bill Details
print("--- Electricity Bill Details ---")
print(f"Energy Charges (EC): $30.00")
print(f> Fixed Charges (FC): $10.00")
print(f"Customer Charges (CC): $0.00")
print(f"Electricity Duty (ED): $1.60 (5%)")
print(f>Total Bill Amount: $21.60")

# Bill Summary for Industrial Customer
print("--- Bill Summary for Industrial Customer ---")
print("New structure: Tiered pricing applied")
```

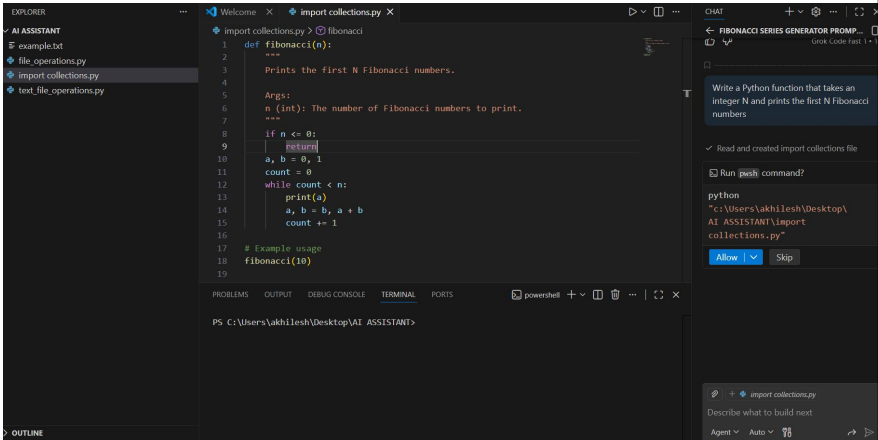
```
File Edit Selection View Go Run Terminal Help ← → Q Assisted coding

# electricity_billings.py
1 # electricity_billings.py
2
3 if customer_type == "residential":
4     customer_charges = 15.0
5 elif customer_type == "commercial":
6     customer_charges = 25.0
7 elif customer_type == "industrial":
8     customer_charges = 40.0
9 else:
10     customer_charges = 0.0
11
12 # Calculate total bill amount
13 total_bill = energy_charges + fixed_charges + customer_charges + electricity_duty
14
15 # Display the input summary
16
17 print("\n--- Meter Reading Summary ---")
18 print(f"Previous units: {prev_units}")
19 print(f"Current units: {curr_units}")
20 print(f"Customer Type: {customer_type.title()}")
21 print(f"Units Consumed: {units_consumed_display}")
22
23 # Display all charges with improved formatting for billing accuracy
24
25 print("\n--- Electricity Bill Details ---")
26 print(f"Energy Charges (EC): {energy_charges:.2f}")
27 print(f"Fixed Charges (FC): {fixed_charges:.2f}")
28 print(f"Customer Charges (CC): {customer_charges:.2f}")
29 print(f"Electricity Duty (ED): {electricity_duty:.2f} ({electricity_duty_rate*100:.0f}% of energy charges)")
30 print(f"Total Bill Amount: {total_bill:.2f}")
31
32 # Real-world applicability notes
33 print("\n--- Bill Summary for (customer_type.title()) Customer ---")
34 print(f"Rate structure: tiered pricing applied")
35 if units_consumed < 0:
36     print(f"Error: Invalid units consumed. Units must be non-negative.")
37 else:
38     print(f"Units Consumed: {units_consumed}")
39
40 # Customer billing accuracy checks
41 if units_consumed < 0:
42     print(f"Warning: Current units are less than previous units. Please verify meter readings.")
43 if energy_charges < 0:
44     print(f"Error: Negative energy charges detected. Check input values.")
45
46
47 # Main execution
48 prev_units = 10
49 curr_units = 25
50 customer_type = "industrial"
51
52 # Meter Reading Summary ---
53 Previous units: 10
54 Current units: 25
55 Customer Type: Industrial
56 Units Consumed: 15
57
58 --- Electricity Bill Details ---
59 Energy Charges (EC): $30.00
60 Fixed Charges (FC): $10.00
61 Customer Charges (CC): $40.00
62 Electricity Duty (ED): $5.00 (33% of energy charges)
63 Total Bill Amount: $85.00
64
65 --- Bill Summary for Industrial Customer ---
66 Rate structure: Tiered pricing applied
```

This program accurately calculates the electricity bill by using basic arithmetic formulas. The code is easy to read because of meaningful variable names and clear print statements. It is applicable in real-world situations as it follows the standard electricity billing structure used by power departments. The formatted output helps users understand each charge clearly.

ID:2303A51510
LAB:3.4

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week2	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 3.4 (Present assignment number)/ 24 (Total number of assignments)			
Q.No.	Question		Expected Time to

		complete
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques Task 1: Zero-shot Prompt – Fibonacci Series Generator Task Description #1 - Without giving an example, write a single comment prompt asking GitHub Copilot to generate a Python function to print the first N Fibonacci numbers.  Expected Output #1 - A complete Python function generated by Copilot without any example provided. - Correct output for sample input N = 7 → 0 1 1 2 3 5 8 - Observation on how Copilot understood the instruction with zero context. Task 2: One-shot Prompt – List Reversal Function Task Description #2 - Write a comment prompt to reverse a list and provide one example below the comment to guide Copilot.	Week2

Expected Output #2

- Copilot-generated function to reverse a list using slicing or loop.
- Output: [3, 2, 1] for input [1, 2, 3]
- Observation on how adding a single example improved Copilot's accuracy.

Task 3: Few-shot Prompt – String Pattern Matching

Task Description #3

- Write a comment with 2–3 examples to help Copilot understand how to check if a string starts with a capital letter and ends with a period.

Expected Output #3

- A function is_valid() that checks the pattern.
- Output: True or False based on input.
- Students reflect on how multiple examples guide Copilot to generate more accurate code.

```
37
38 # Write a function is_valid(s) that checks if a string starts with a capital
39 # Example 1: is_valid("Hello world.") should return True
40 # Example 2: is_valid("hello world.") should return False
41 # Example 3: is_valid("Hello world") should return False
42
43 def is_valid(s):
44     """
45     Checks if the string starts with a capital letter and ends with a period.
46
47     Args:
48     s (str): The string to check.
49
50     Returns:
51     bool: True if valid, False otherwise.
52     """
53     if not s:
54         return False
55     return s[0].isupper() and s.endswith('.')
56
57 # Example usage
58 print(is_valid("Hello world.")) # True
59 print(is_valid("hello world.")) # False
60 print(is_valid("Hello world")) # False
61
```

Task 4: Zero-shot vs Few-shot – Email Validator

Task Description #4

- First, prompt Copilot to write an email validation function using zero-shot (just the task in comment).
- Then, rewrite the prompt using few-shot examples.

```
69 # Example 1: is_valid_email("user@example.com") should return True
70 # Example 2: is_valid_email("user@") should return False
71 # Example 3: is_valid_email("userexample.com") should return False
72
73 def validate_email(email):
74     """
75     Detailed email validation using few-shot prompt.
76
77     Args:
78     email (str): The email address to validate.
79
80     Returns:
81     bool: True if valid, False otherwise.
82     """
83     if not email or '@' not in email:
84         return False
85     local, domain = email.split('@', 1)
86     if not local or not domain or '.' not in domain:
87         return False
88     return True
89
90 # Example usage
91 print(validate_email_zero_shot("user@example.com")) # True
92 print(validate_email_zero_shot("invalid")) # False
93
94 print(validate_email_few_shot("user@example.com")) # True
95 print(validate_email_few_shot("user@")) # False
96 print(validate_email_few_shot("userexample.com")) # False
97
```

Expected Output #4

- Compare both outputs:
Zero-shot may result in basic or generic validation.
Few-shot gives detailed and specific logic (e.g., @ and domain checking).
- Submit both code versions and note how few-shot improves reliability.

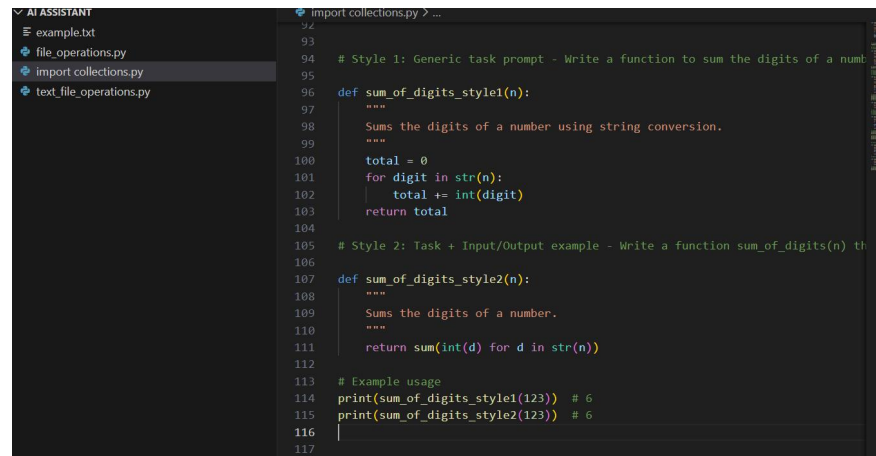
Task 5: Prompt Tuning – Summing Digits of a Number

Task Description #5

- Experiment with 2 different prompt styles to generate a function that returns the sum of digits of a number.

Style 1: Generic task prompt

Style 2: Task + Input/Output example



```
92
93
94 # Style 1: Generic task prompt - Write a function to sum the digits of a num
95
96 def sum_of_digits_style1(n):
97     """
98     Sums the digits of a number using string conversion.
99     """
100     total = 0
101     for digit in str(n):
102         total += int(digit)
103     return total
104
105 # Style 2: Task + Input/Output example - Write a function sum_of_digits(n) th
106
107 def sum_of_digits_style2(n):
108     """
109     Sums the digits of a number.
110     """
111     return sum(int(d) for d in str(n))
112
113 # Example usage
114 print(sum_of_digits_style1(123)) # 6
115 print(sum_of_digits_style2(123)) # 6
116
117
```

Expected Output #5

- Two versions of the `sum_of_digits()` function.
- Example Output: `sum_of_digits(123) → 6`
- Short analysis: which prompt produced cleaner or more optimized code and why?

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots